Passive Radar User Manual

KrakenSDR Multi-Beam Bistatic Radar

Simon Knight

Adelaide, Australia

March 2026

> **About this manual.** This manual covers deploying the KrakenSDR passive radar system, configuring beams, tuning CFAR detection, tracking targets, correlating radar tracks with ADS-B aircraft data, and recording sessions to HDF5 for offline analysis. It assumes familiarity with software-defined radio concepts and basic Linux administration.

Contents

1 Quick Start

Get detections on screen in under ten minutes.

Step 1. Connect the KrakenSDR to the Raspberry Pi via USB, then attach the five antennas: one reference antenna directed at the transmitter of interest, and four surveillance antennas covering the desired surveillance sectors.

Step 2. Clone the repository and enter the project directory:

```
git clone https://github.com/example/passive-radar.git
cd passive-radar
```

Step 3. Copy the example environment file and populate the mandatory geometry fields with your receiver and transmitter coordinates:

```
cp .env.example .env
$EDITOR .env
```

At a minimum, set `PASSIVE_RADAR_GEOMETRY_RX_LAT`, `PASSIVE_RADAR_GEOMETRY_RX_LON`, `PASSIVE_RADAR_GEOMETRY_TX_LAT`, and `PASSIVE_RADAR_GEOMETRY_TX_LON`.

Step 4. Start the full system using `uv`:

```
uv run python run_server.py
```

The server auto-discovers the Raspberry Pi via mDNS. If discovery fails, pass the Pi's IP address explicitly:

```
uv run python run_server.py --pi-ip 192.168.1.50
```

Step 5. Open the web dashboard in a browser:

```
http://localhost:8080
```

Step 6. Verify the system is working:

- The Range-Doppler maps for all four beams should be updating at roughly 2 Hz on the dashboard.
- Bright peaks in the maps that move across frames indicate targets. A stationary bright band at zero Doppler is residual direct-path clutter—this is normal.
- The status bar should show "`streaming`" (not "`connecting`").

2 Installation & Prerequisites

2.1 Hardware Requirements

▷ **Prerequisites**

- KrakenSDR five-channel coherent SDR receiver (Kraken RTL-SDR variant with hardware phase synchronisation)
- Raspberry Pi 4 (4 GB RAM recommended) running Raspberry Pi OS Bookworm (64-bit), with the Heimdall DAQ firmware installed
- One directional reference antenna (Yagi or log-periodic) aimed at the chosen transmitter
- Four surveillance antennas covering the desired angular sectors (omnidirectional or low-gain directional antennas)
- Local network connection between the Raspberry Pi and the processing workstation (gigabit Ethernet or strong Wi-Fi)

Tip

The reference antenna is the most important element of the antenna system. Use a directional antenna (Yagi, log-periodic, or dish) aimed at the transmitter to maximise reference SNR. A poor reference signal directly limits clutter cancellation depth and therefore target visibility.

Caution

The system requires unobstructed line-of-sight from the receiver to the transmitter for the reference channel. Buildings, terrain, and dense vegetation between the receiver and the transmitter degrade the reference signal. The direct-path excess loss should not exceed 15 dB above free-space loss for reliable Wiener cancellation.

2.2 Software Prerequisites

▷ **Prerequisites**

- Python 3.9 or later (Python 3.10 recommended for performance)
- `uv` package manager (`pip install uv`)
- NumPy, SciPy — core numerical routines
- Plotly / Dash — web dashboard
- FilterPy — Kalman filter implementation
- pyAPRiL — Wiener SMI-MRE clutter cancellation and FFT-based cross-ambiguity function computation
- `pydantic` and `pydantic-settings` — configuration validation
- `h5py` — HDF5 recording (optional, for data capture)
- `dump1090` running locally — ADS-B ground truth (optional)

2.3 Installing Dependencies

Install all Python dependencies in one step:

```
uv sync
```

This reads `pyproject.toml` and creates a virtual environment under `.venv/`.

Tip

`uv` is dramatically faster than `pip` for resolving large dependency graphs. Use `uv run python ...` to invoke any script with the managed environment active, without manually activating the virtual environment.

2.4 Installing the Heimdall DAQ Firmware on the Raspberry Pi

The Raspberry Pi runs the Heimdall DAQ firmware, which drives the KrakenSDR hardware and streams IQ frames to the workstation. Follow the KrakenSDR documentation to install Heimdall on the Pi before proceeding. Once installed, Heimdall should start automatically and advertise itself via mDNS as `_krakendaq._tcp`.

3 Core Workflows

3.1 Initial Deployment

Spend time on antenna placement. A 3 dB improvement in reference SNR is worth more than any software tuning.

Step 1. **Position the antennas.** Mount the reference antenna at a clear vantage point aimed at the transmitter. Distribute the four surveillance antennas to cover the desired angular sectors. Typical sectors are 90° wide, giving 360° coverage with four beams. Minimise cable run lengths; coaxial loss at VHF/UHF reduces sensitivity.

Step 2. **Connect to the KrakenSDR.** Plug all five antenna feeds into the KrakenSDR's SMA ports (port 0 for reference, ports 1–4 for surveillance). Connect the KrakenSDR to the Raspberry Pi via USB. Power the KrakenSDR from a stable 5 V supply rated for at least 2 A.

Step 3. **Configure `.env`.** Edit the `.env` file to set your deployment geometry and desired beam azimuths:

```
# Geometry: receiver site
PASSIVE_RADAR_GEOMETRY_RX_LAT=-34.9285
PASSIVE_RADAR_GEOMETRY_RX_LON=138.6007
PASSIVE_RADAR_GEOMETRY_RX_ALT=50.0

# Geometry: transmitter (e.g., an FM broadcast tower)
PASSIVE_RADAR_GEOMETRY_TX_LAT=-34.7600
PASSIVE_RADAR_GEOMETRY_TX_LON=138.6333
PASSIVE_RADAR_GEOMETRY_TX_ALT=300.0

# Network
PASSIVE_RADAR_PI_IP=192.168.1.50
PASSIVE_RADAR_LISTEN_PORT=5000
```

Step 4. **Set beam azimuths.** After starting the server, navigate to the Beam Geometry panel on the dashboard and set the azimuth and beam width for each of the four beams. These values are persisted to `output/beam_geometry.json` and survive restarts.

Step 5. **Start the server** and verify the dashboard shows live Range-Doppler maps as described in Section 1.

3.2 Real-Time Detection Monitoring

Step 1. Open the dashboard at `http://localhost:8080`.

Step 2. The main panel shows a 2×2 grid of Range-Doppler maps, one per beam. The horizontal axis is bistatic range (bins); the vertical axis is Doppler frequency (bins).

Step 3. Targets appear as bright peaks away from the zero-Doppler line. The zero-Doppler band (typically five bins wide) is blanked to remove residual stationary clutter.

Step 4. CFAR detections are overlaid as markers on the map. Each marker shows the detection's SNR. A busy airspace should yield one or more moving detections per beam at any given time.

Step 5. The Tracks panel shows confirmed tracks with their track ID, beam, status (TENTATIVE or CONFIRMED), estimated position, and ADS-B callsign (if correlated).

Step 6. The Map panel renders confirmed tracks as geographic positions overlaid on a background map.

Step 7. Watch the frame counter in the status bar to verify the system is processing frames at the expected rate (≈65–80 FPS from the Heimdall DAQ).

3.3 CFAR Threshold Tuning

Start with the default `Pfa = 1e-6` and increase it (loosen the threshold) only if you are missing obvious targets.

CA-CFAR is the primary detection algorithm. Its parameters directly trade off sensitivity against false alarm rate.

Step 1. On the dashboard, expand the **CFAR Controls** panel for the beam you wish to tune.

Step 2. **Guard cells** (`guard_cells_range`, `guard_cells_doppler`): the exclusion zone around the cell under test. Increase these if strong targets are "eating" their own noise estimate and suppressing nearby detections. Default: 3.

Step 3. **Training cells** (`training_cells_range`, `training_cells_doppler`): the annulus used to estimate the noise floor. Larger training windows give a more stable estimate but may span regions of non-uniform noise. Default: 10.

Step 4. **Probability of false alarm** (`pfa`): lower values raise the threshold, reducing false alarms at the cost of sensitivity. Typical values: `1e-6` (default, conservative), `1e-5` (moderate), `1e-4` (aggressive, many false alarms).

Step 5. **Zero-Doppler blanking** (`zero_doppler_bins`): bins suppressed around zero Doppler. Increase if stationary clutter is producing excessive false alarms near the zero-Doppler line. Default: 5.

Step 6. Apply the changes using the **Update** button. The new parameters take effect on the next frame; no restart is required.

Step 7. Observe the detection overlay on the Range-Doppler map. A well-tuned system shows sparse detections (under ten per beam per frame) with clear target trajectories visible over time.

> **Tip**
>
> If you are seeing many detections that do not form coherent tracks, reduce `pfa` (tighten the threshold) or increase `zero_doppler_bins`. If confirmed targets are being lost mid-track, loosen the threshold or reduce `delete_threshold` (allow more consecutive misses before track deletion).

3.4 Target Tracking

The tracking subsystem maintains persistent target identities across frames using a four-dimensional Kalman filter and Global Nearest Neighbour (GNN) data association.

Step 1. **Track initialisation.** Every CFAR detection that cannot be associated with an existing track spawns a new TENTATIVE track. Tentative tracks are shown on the dashboard only if `show_tentative_tracks` is enabled (disabled by default).

Step 2. **Track confirmation.** A track transitions from TENTATIVE to CONFIRMED when it accumulates at least `confirm_m` detections within the last `confirm_n` frames (default: 2-of-3). Confirmed tracks are displayed on the Tracks panel and the map.

Step 3. **Track maintenance.** Each new frame, the Kalman filter predicts each track's position, and the GNN algorithm associates incoming detections to the closest track within the

Mahalanobis distance gate (`gate_threshold`, default: 9.21, the 99% χ^2 threshold for two degrees of freedom).

Step 4. Track deletion. A confirmed track is deleted when it accumulates `delete_threshold` consecutive frames without an associated detection (default: 5). The track is then removed from all outputs.

Step 5. Tuning the Kalman filter. The process noise parameter `process_noise_q` controls how quickly the filter adapts to manoeuvring targets. Increase it (e.g., to 2.0) for fast-manoeuvring targets; decrease it (e.g., to 0.1) in low-clutter environments where smooth tracks are preferred.

Step 6. Applying a tuning profile. Three preset profiles are available:

- `default` — balanced settings for civil airspace
- `high_multipath` — wider guard cells, longer range
- `high_speed` — elevated process noise for fast targets

Set the profile via the environment variable `PASSIVE_RADAR_TUNING_PROFILE=high_speed` or the dashboard dropdown.

3.5 ADS-B Correlation

ADS-B correlation enriches radar tracks with aircraft identity information and provides a ground-truth reference for system validation.

Step 1. Install and start `dump1090`. On the same machine as the radar server (or on a machine accessible over the network), install and start `dump1090-fa` or compatible:

```
sudo systemctl start dump1090-fa
```

Verify that `http://localhost:8080/data/aircraft.json` returns a JSON list of aircraft.

> **Tip**
>
> `dump1090` uses port 8080 by default, the same as the radar dashboard. Either run dump1090 on a separate machine or change the radar dashboard port via `PASSIVE_RADAR_DASHBOARD_PORT=8081`.

Step 2. Enable ADS-B integration. Add the following to `.env`:

```
PASSIVE_RADAR_ADSB_ENABLED=true
PASSIVE_RADAR_ADSB_URL=http://localhost:8080/data/aircraft.json
PASSIVE_RADAR_ADSB_UPDATE_INTERVAL=1.0
```

Step 3. Restart the server to pick up the new configuration.

Step 4. Validate correlation. On the dashboard Tracks panel, confirmed radar tracks that have been correlated with ADS-B aircraft display the aircraft callsign, ICAO24 hex code, and barometric altitude. Compare the radar-projected position on the map with the ADS-B position to assess system accuracy.

Step 5. Interpret correlation results. A radar track position within 5 km of an ADS-B aircraft position, with consistent Doppler and heading, is considered a successful correlation. Typical agreement is within 10–20% of the true position; residual error arises from beam width (±2–5°) and bistatic range estimation uncertainty.

> **Caution**
>
> ADS-B correlation is one-to-one: each radar track is matched to at most one aircraft. In dense airspace with many aircraft in the same beam and range cell, correlation may be ambiguous. Do not rely on ADS-B callsign assignment for safety-critical applications.

3.6 Data Recording and Playback

The recording subsystem captures detection data to HDF5 files for offline analysis, algorithm development, and system validation.

Step 1. Enable recording. Add to `.env`:

```
PASSIVE_RADAR_RECORDING_ENABLED=true
PASSIVE_RADAR_RECORDING_DIRECTORY=./output/recordings
PASSIVE_RADAR_RECORDING_FILENAME_PREFIX=radar_session
```

Step 2. Configure file rotation. The recording system rotates to a new file when either a time or size limit is reached:

```
PASSIVE_RADAR_RECORDING_ROTATE_TIME_S=900
PASSIVE_RADAR_RECORDING_ROTATE_SIZE_MB=512
```

Set either to `0` to disable that rotation criterion.

Step 3. Start the server. Recording begins automatically on the first detection. Files are named `radar_session_YYYYMMDD_HHMMSS.h5` in the recording directory.

Step 4. Monitor recording status. The dashboard status bar shows the current recording filename, the number of frames written, and the approximate file size.

Step 5. Offline playback. To replay a recorded session, configure the playback subsystem:

```
PASSIVE_RADAR_PLAYBACK_ENABLED=true
```

Then open the Playback panel on the dashboard, select a recording file, and use the transport controls (play, pause, seek). Two playback modes are available:

- **Recorded tracks** — replays pre-computed tracks directly from the HDF5 file; fast, no reprocessing.
- **Recompute tracks** — re-runs the full DSP and tracking pipeline on the recorded IQ data; allows tuning parameters and re-evaluating detections.

Step 6. Inspect HDF5 files directly with any HDF5 viewer or Python:

```
python -c "import h5py; f = h5py.File('radar_session.h5'); print(list(f.keys()))"
```

4 Configuration Reference

All configuration is loaded from a `.env` file in the project root. Every variable uses the prefix `PASSIVE_RADAR_` followed by the config-group name (for nested models, separated by underscores).

4.1 Environment Variable Format

Nested Pydantic models are addressed by chaining group names:

```
PASSIVE_RADAR_<GROUP>_<FIELD>=<value>
```

For example, to set the receiver latitude in `GeometryConfig`:

```
PASSIVE_RADAR_GEOMETRY_RX_LAT=-34.9285
```

4.2 Configuration Groups

4.2.1 ServerConfig

Top-level server configuration.

Variable	Default	Description
`PI_IP`	*auto*	Raspberry Pi IP address (mDNS if unset)
`LISTEN_PORT`	`5000`	UDP port for IQ data stream
`CONTROL_PORT`	`5001`	TCP port for control channel
`OUTPUT_DIR`	`./output`	Output directory for Range-Doppler files
`QUEUE_SIZE`	`32`	IQ frame queue depth (frames)
`STATUS_INTERVAL`	`5.0`	Status log interval (seconds)
`USE_MDNS`	`true`	Enable mDNS Pi discovery
`VERBOSE`	`false`	Enable debug logging

4.2.2 BeamConfig

Per-beam signal processing parameters. Beam fields are indexed as `PASSIVE_RADAR_BEAM_0_<FIELD>` through `PASSIVE_RADAR_BEAM_3_<FIELD>`.

Field	Default	Description
`surveillance_channel`	`1–4`	IQ channel index (0 is reserved for reference)
`azimuth`	`0.0`	Beam centre azimuth, degrees true north
`beam_width`	`30.0`	Full beam width in degrees
`max_bistatic_range`	`128`	Maximum range in samples
`max_doppler`	`256`	Maximum Doppler bins
`colormap`	`viridis`	Plotly colourscale name
`dynamic_range_min`	`-60.0`	Colourmap minimum (dB)
`dynamic_range_max`	`0.0`	Colourmap maximum (dB)
`persistence`	`1`	Frame averaging count
`enabled`	`true`	Activate this beam
`clean_enabled`	`false`	Enable CLEAN sidelobe suppression
`spatial_filter_enabled`	`false`	Enable MVDR spatial filtering

4.2.3 CFARConfig

CA-CFAR detection parameters (nested under each `BeamConfig`).

Field	Default	Description
`guard_cells_range`	`3`	Guard cells in range dimension
`guard_cells_doppler`	`3`	Guard cells in Doppler dimension
`training_cells_range`	`10`	Training cells in range dimension
`training_cells_doppler`	`10`	Training cells in Doppler dimension
`pfa`	`1e-6`	Probability of false alarm
`zero_doppler_bins`	`5`	Bins suppressed at zero Doppler
`max_detections`	`50`	Peak detections per frame (by SNR)

4.2.4 TrackConfig

Kalman filter and track lifecycle parameters (nested under each `BeamConfig`).

Field	Default	Description
`gate_threshold`	`9.21`	Mahalanobis gate (99% χ^2, 2 DOF)
`confirm_m`	`2`	Hits required for confirmation
`confirm_n`	`3`	Window size for M-of-N confirmation
`delete_threshold`	`5`	Consecutive misses before deletion
`process_noise_q`	`0.5`	Kalman process noise spectral density
`dt`	`0.5`	Expected frame interval (seconds)
`trail_length`	`50`	Track trail history length (frames)
`show_tentative_tracks`	`false`	Show unconfirmed tracks on dashboard

4.2.5 GeometryConfig

Bistatic radar geometry for geographic projection.

Field	Default	Description
`RX_LAT`	`0.0`	Receiver latitude (decimal degrees)
`RX_LON`	`0.0`	Receiver longitude (decimal degrees)
`RX_ALT`	`0.0`	Receiver altitude (m above sea level)
`TX_LAT`	`0.0`	Transmitter latitude (decimal degrees)
`TX_LON`	`0.0`	Transmitter longitude (decimal degrees)
`TX_ALT`	`0.0`	Transmitter altitude (m above sea level)

4.2.6 ADSBConfig

ADS-B integration for ground-truth aircraft correlation.

Field	Default	Description
`ENABLED`	`false`	Enable ADS-B correlation subsystem
`URL`	`http://localhost:8080/data/aircraft.json`	Feed URL
`UPDATE_INTERVAL`	`1.0`	Fetch interval (seconds)

4.2.7 RecordingConfig

HDF5 detection recording.

Field	Default	Description
`ENABLED`	`false`	Enable recording subsystem
`DIRECTORY`	`./output/recordings`	HDF5 output directory
`FILENAME_PREFIX`	`radar_session`	File name prefix
`ROTATE_TIME_S`	`900`	Rotate after N seconds (0 = off)
`ROTATE_SIZE_MB`	`512`	Rotate after N MB (0 = off)
`QUEUE_MAX`	`100`	Write queue depth (drops on overflow)
`COMPRESSION`	`lzf`	HDF5 compression filter

4.3 Minimal `.env` Example

A minimal configuration for a single-site deployment in Adelaide, Australia, using a local FM transmitter:

```
# Receiver site -- Waite campus
```

```
PASSIVE_RADAR_GEOMETRY_RX_LAT=-34.9694
PASSIVE_RADAR_GEOMETRY_RX_LON=138.6333
PASSIVE_RADAR_GEOMETRY_RX_ALT=120.0

# Transmitter -- Mount Lofty FM broadcast tower
PASSIVE_RADAR_GEOMETRY_TX_LAT=-34.9770
PASSIVE_RADAR_GEOMETRY_TX_LON=138.7107
PASSIVE_RADAR_GEOMETRY_TX_ALT=727.0

# Network
PASSIVE_RADAR_PI_IP=192.168.1.50
PASSIVE_RADAR_LISTEN_PORT=5000

# Enable ADS-B for validation
PASSIVE_RADAR_ADSB_ENABLED=true

# Enable HDF5 recording
PASSIVE_RADAR_RECORDING_ENABLED=true
```

5 Entry Points

The system provides three entry-point scripts.

Command Reference

Command	Description
`run_server.py`	Full system: IQ receiver, DSP, tracking, dashboard, ADS-B
`run_headless.py`	Processing only: no dashboard, suitable for headless servers
`run_dashboard.py`	Dashboard only: connect to a running server instance

run_server.py Options

Command	Description
`--pi-ip <IP>`	Override mDNS discovery with a fixed Pi IP address
`--verbose`	Enable debug-level logging to stdout
`--mock`	Use synthetic IQ data (no Heimdall DAQ required)
`--frames <N>`	Process only N frames then exit (for testing)

run_headless.py Options

Command	Description
`--pi-ip <IP>`	Override mDNS discovery with a fixed Pi IP address
`--verbose`	Enable debug-level logging
`--mock`	Synthetic IQ data mode
`--frames <N>`	Exit after N frames

run_dashboard.py Options

Command	Description
`--server-ip <IP>`	IP address of the running server (default: localhost)
`--port <N>`	Dashboard HTTP port (default: 8080)

Tip

Use `--mock` during development and algorithm tuning to exercise the full pipeline without requiring the Raspberry Pi and KrakenSDR hardware. The mock data generator produces synthetic chirp-like targets at randomised range and Doppler that exercise the CFAR detector and tracking subsystem.

6 Signal Processing Concepts

This section provides the minimum signal processing background for practitioners deploying and tuning the system.

6.1 Bistatic Radar Geometry

In bistatic passive radar, the transmitter $\mathbf{t}$ and receiver $\mathbf{r}$ are at different geographic locations. A target at position $\mathbf{p}$ reflects the transmitted signal, and the total path length travelled is the *bistatic range*:

$$R_b = |\mathbf{p} - \mathbf{t}| + |\mathbf{p} - \mathbf{r}| - |\mathbf{t} - \mathbf{r}|$$

The baseline term $|\mathbf{t} - \mathbf{r}|$ is subtracted so that the direct-path signal registers at $R_b = 0$. The locus of constant R_b is an ellipsoid with the transmitter and receiver as foci. A single receiver beam constrains the target to the intersection of this ellipsoid and the beam cone; without additional information the target position has significant angular ambiguity.

6.2 Cross-Ambiguity Function (CAF)

The CAF measures the correlation between the reference signal $r(t)$ and a time-delayed, Doppler-shifted copy of the surveillance signal $s(t)$:

$$\mathrm{CAF}(\tau, f_d) = \int_0^T s(t)\, r^*(t - \tau)\, e^{-j2\pi f_d t}\, \mathrm{d}t$$

Computing the CAF over all delay–Doppler pairs yields the Range-Doppler map. The system uses an FFT-based batch implementation (via pyAPRiL's `cc_detector_ons`) with typical latency of 30–60 ms per beam for 2048 Doppler bins and 128 range bins.

6.3 Wiener Clutter Cancellation

The surveillance channel receives a strong direct-path copy of the transmitter signal, which can exceed weak target echoes by 40–60 dB. The Wiener SMI-MRE filter estimates the direct-path component and subtracts it before the CAF is computed. Given reference $\mathbf{r} \in \mathbb{C}^N$ and surveillance $\mathbf{x} \in \mathbb{C}^N$, the filter computes:

$$\hat{\alpha} = \frac{\mathbf{r}^H \mathbf{x}}{\mathbf{r}^H \mathbf{r} + \epsilon}, \qquad \mathbf{x}_{\text{clean}} = \mathbf{x} - \hat{\alpha}\, \mathbf{r}$$

In practice, a multi-tap version models multipath via sample matrix inversion (provided by pyAPRiL). Cancellation depth is typically 30–50 dB.

6.4 CA-CFAR Detection

Cell-Averaging CFAR adapts the detection threshold to the local noise environment. For each cell under test at (d, r), the algorithm:

1. Excludes a guard region ($G_r \times G_d$ cells) around the test cell.
2. Estimates the noise floor from a surrounding training region ($T_r \times T_d$ cells).
3. Computes the threshold as $\eta = \hat{P}_n \cdot \alpha_{\text{CFAR}}$, where α_{CFAR} is derived from the desired P_{fa}.
4. Declares a detection if the cell exceeds η.

Sub-bin peak position is refined via three-point parabolic interpolation.

6.5 Kalman Filter State Model

Each track carries a four-dimensional constant-velocity state: $\mathbf{x} = [\text{range},\ \dot{r},\ f_d,\ \dot{f}_d]^T$. The measurement vector is $\mathbf{z} = [\text{range}_m,\ f_d]^T$ from the CFAR detector. Process noise uses a continuous white-noise acceleration model scaled by `process_noise_q`. Data association uses the Global Nearest Neighbour algorithm with Mahalanobis distance gating.

7 Troubleshooting

Problem 1: No detections appear on any beam

Cause: Several causes are possible: the KrakenSDR is not streaming, the reference signal is too weak, or the CFAR threshold is too tight.

Solution: Check the status bar — it should show "streaming", not "connecting". Verify the Heimdall DAQ is running on the Pi. If streaming, check the raw Range-Doppler map for any structure; if completely flat or random, the reference signal may be too weak or absent. Try loosening the threshold by increasing `pfa` to `1e-4` temporarily. If the raw map shows structure but no CFAR markers, increase `pfa` or decrease `training_cells_range` and `training_cells_doppler`.

Problem 2: Tracks disappear after a few frames (failing to confirm)

Cause: The 2-of-3 confirmation criterion is not being met, usually because the detection rate on weak targets is below 67%.

Solution: Loosen the CFAR threshold (`pfa`), or reduce `confirm_m` to 1 (confirm on first detection). Also check that the target's Doppler is not falling within the zero-Doppler blanking window (`zero_doppler_bins`); slow-moving targets near zero Doppler may be suppressed.

Problem 3: Tracking is per-beam only — no cross-beam fusion

Cause: This is a known system limitation, not a fault. Tracks are maintained independently per beam; a single aircraft crossing two beam sectors will appear as two separate tracks with different IDs.

Solution: Cross-beam fusion is not yet implemented. Use ADS-B correlation (Section 3.5) to associate separate-beam tracks with the same aircraft via callsign. This is tracked in the technical report PR-TR-001, Section 8.

Problem 4: Bistatic range estimates are 10–20% off compared with ADS-B ground truth

Cause: This is the expected accuracy of the bistatic projection. Residual error arises from antenna beam width (±2–5° angular ambiguity) and imperfect clutter cancellation biasing the range estimate.

Solution: Verify that `GeometryConfig` coordinates are accurate (use GPS, not map estimation). Validate receiver and transmitter altitude values. For research-grade accuracy, additional receiver sites or altitude constraints from ADS-B are required to resolve the bistatic range–angle ambiguity.

Problem 5: Excessive false alarms — dashboard is cluttered with spurious detections

Cause: The CFAR threshold is too loose, or there is strong multipath or interference in the surveillance channel.

Solution: Tighten `pfa` (e.g., reduce from `1e-5` to `1e-6`). Increase `zero_doppler_bins` if the false alarms cluster near zero Doppler. Consider enabling the CLEAN algorithm (`clean_enabled=true`) to suppress sidelobes from strong stationary clutter that survive Wiener cancellation. Check antenna placement: surveillance antennas should not be co-located with the reference antenna, as direct breakthrough degrades cancellation.

Problem 6: Reference signal quality is poor — clutter cancellation depth is below 20 dB

Cause: The reference antenna is receiving a multipath-contaminated signal or is pointed away from the transmitter.

Solution: Re-align the reference antenna. Check for physical obstructions between the reference antenna and the transmitter tower. Use a spectrum analyser to verify the signal level at the reference port. A high-gain directional antenna (Yagi or dish) on the reference channel significantly improves cancellation depth.

Problem 7: CPU usage pegs at 100% and the UI becomes unresponsive

Cause: Range-Doppler processing is compute-heavy and can saturate a single CPU core (or the whole machine) depending on FFT size, beam count, and enabled features (Wiener, CLEAN, tracking, ADS-B correlation). If the UI thread shares the same event loop, rendering will lag. Some browsers also struggle with high-rate WebSocket updates.

Solution: Reduce the update rate (lower dashboard FPS), disable optional modules temporarily (set `clean_enabled=false`, disable ADS-B correlation), and confirm you are not running both the dashboard and offline analysis on the same host. If available, enable the GPU CAF path and verify it is actually selected in the logs/metrics.

Problem 8: KrakenSDR hardware lock-in: cannot switch to a different SDR

Cause: The system's IQ streaming protocol and 1024-byte header format are tied to the Heimdall DAQ firmware, which is specific to the KrakenSDR hardware.

Solution: Alternative SDR hardware (e.g., USRP, HackRF) requires a different streaming adapter. The `_receiver/iq_header.py` module defines the Heimdall header format; a compatible adapter for another DAQ system must produce the same header structure. The `rtltcp-rust` project (Section 8) provides an abstraction layer under active development.

Problem 9: Manual CFAR threshold override needed — `pfa` does not produce the expected behaviour

Cause: The CFAR threshold factor is computed from `pfa` and the number of training cells. In highly non-Gaussian noise environments (strong interference, near-field clutter) the theoretical P_{fa} does not match empirical false alarm rates.

Solution: Use the `threshold_factor` override to set the threshold multiplier directly, bypassing the P_{fa} computation. Set `PASSIVE_RADAR_BEAM_0_CFAR_THRESHOLD_FACTOR=12.0` (for example) and tune empirically by observing the detection rate on known traffic.

8 Integration with Other Tools

The passive radar system is designed as a standalone application but interoperates with several external tools.

8.1 Spectra — SDR Spectrum Intelligence

Spectra provides shared SDR infrastructure and wide-band spectrum intelligence across the same KrakenSDR hardware. When running alongside the passive radar system, Spectra handles spectrum monitoring duties while the radar occupies a fixed centre frequency for coherent processing.

```
# Start Spectra on a separate SDR device
spectra run --device rtlsdr --port 7000
```

See also: *Spectra User Manual* (SPECTRA-UM-001) — Shared SDR device management and spectrum intelligence workflows.

8.2 dump1090 — ADS-B Feed

The passive radar system consumes aircraft position data from a local `dump1090` (or compatible `readsb`) instance via the `aircraft.json` HTTP endpoint. See Section 3.5 for configuration details.

```
# Verify dump1090 feed is available
curl http://localhost:8080/data/aircraft.json | python -m json.tool | head -40
```

See also: *dump1090-fa* documentation — `aircraft.json` schema and feed configuration.

8.3 rtltcp-rust — SDR Server Abstraction

The `rtltcp-rust` project provides a hardware abstraction layer for RTL-SDR and KrakenSDR receivers, exposing a standardised TCP streaming interface. It is under active development as a future replacement for the direct Heimdall DAQ integration, enabling the passive radar system to work with any SDR hardware that supports the rtl-tcp protocol.

```
# Start rtltcp-rust in KrakenSDR compatibility mode
rtltcp-rust --device kraken --port 1234
```

See also: *rtltcp-rust* README — Hardware compatibility matrix and streaming protocol specification.

9 Further Reading

- *KrakenSDR Passive Radar System — Technical Reference* (PR-TR-001) — Architecture, signal processing algorithm design rationale, Kalman filter mathematics, advanced algorithms (CLEAN, MVDR, micro-Doppler), and performance benchmarks.